

Lab 4: ROM Based Access Control on FPGA Board

Abdur Aziz

Course Number : ECE 5440

System Description:

The **Digit Timer system** is a hardware-based timing control module implemented on an FPGA board. The system functions as a precise clock divider, converting the high-frequency system clock into slower, human-readable timing pulses. These pulses are used to regulate the refresh rate of the seven-segment display and control the pacing of the game logic. The module counts a predefined number of clock cycles and generates a timing pulse upon reaching the terminal count, effectively synchronizing display updates and user interactions. By providing consistent and accurate timing signals, the Digit Timer ensures smooth operation of the game, maintaining proper flow between input handling, display transitions, and gameplay progression.

The **Access Controller system** is a hardware-based digital security module implemented on an FPGA board. The system simulates a secure access point, where a user must enter a 4-digit numeric passcode using physical push-buttons. The module compares the entered passcode to a predefined access code, and upon a correct match, the system grants access by displaying a success message on the seven-segment display and activating status indicators.

However, for Lab 4, the access controller is split up into the Game Controller Module and the Authenticator Module as per the instructions. The authentication process builds upon the secure password mechanism introduced in Lab 2, but is now optimized for a single-player environment. In this setup, the player must enter a predefined four-digit passcode, “2062,” using physical input switches on the FPGA board. Each digit is passed through a Button Shaper module to eliminate mechanical switch noise and stabilize the input signal. These sanitized values are captured sequentially by the Load Register module.

Once all four digits are entered, the Authentication Module compares the player's input against a preset password stored in a ROM. If the passcode matches, a LoggedIn

signal is asserted and passed to the Game Controller Module, which then transitions the system into gameplay by enabling game control modules such as the timer and RNG. If the input is incorrect, the system remains in a locked state, prompting the player to retry authentication. Throughout the process, the Seven-Segment Decoder module provides visual feedback by displaying the user's current inputs and system states on the board's seven-segment displays. Additionally, the system uses an Adder module to interpret and process button signals into usable numeric values internally.

Once the user has successfully logged in, then the timer will read “99”. It will remain stagnant unless the player presses the “**Game Start**” button again. When it's pressed, then the timer will start and will decrement by 1 second. During this time, the player can add numbers by using the player input switches and by pressing the random number generator button. It's important to note that the button will generate hexadecimal values, 0-9 for single digits, and A-F for two digits. The sum displayed is the player's input number plus the random number generated.

Figure 1.1 below depicts the toggle switches and buttons for loading in the user's last four digits of their People Soft ID and the switches used to input the PeopleSoft ID password. If the entered password has been successful, the “**Logged In**” LED will display, and users can play the game. Otherwise, the “**Logged Out**” LED will remain on. It's important to note that by default, the “**Logged Out**” LED light will be on when the FPGA board is turned on. Additionally, should the user press the “**Reset**” button, the “**Logged Out**” LED will turn on and the “**Logged In**” LED will turn off. ***It's important to note that in Lab 3, the Logout feature was not supposed to be implemented. However, for the sake of debugging hardware and meticulous code clarity analyzing, it was included.**

FPGA Board:

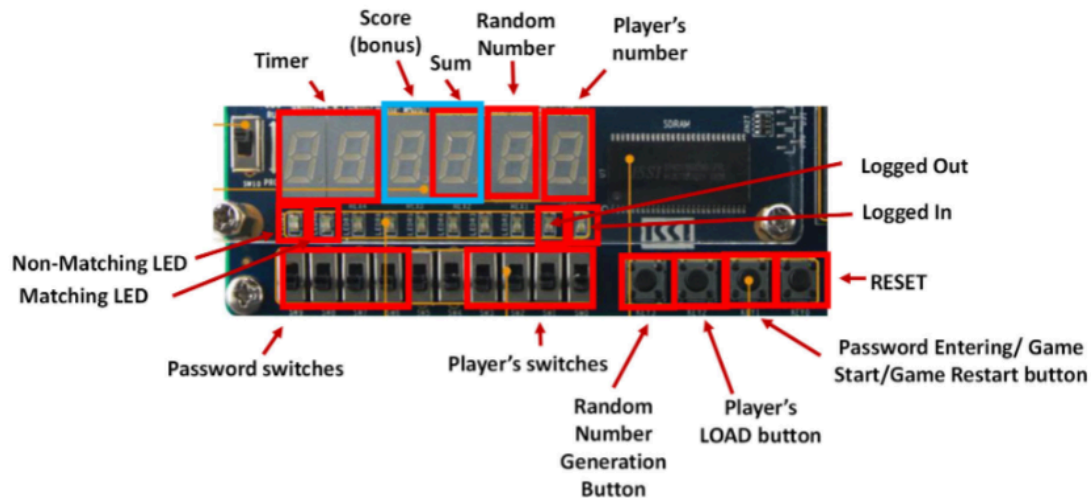


Figure 1.1 FPGA Password Entering Switches, Random Number Generator Button, and Digit Timer Display

Figure 1.2 below shows the FPGA Board, captured from the DE0-CV user manual, which was leveraged to operate the mental math binary game. Starting from right to left with respect to Figure 1.1, the rightmost 4 bit toggle switches are used to input Player 1's binary number. The rightmost 3 segment display is Player 1's number display, as indicated by "**Player 1 display**". Starting from left to right with respect to Figure 1.1, the leftmost first four toggle bits are used to input the password to enter to play the game. The leftmost 3 segment display is the Digit Timer, which is indicated by "**Timer**". The sum is displayed in the middle 3 segment display, as indicated by "**Sum display**". The display after the sum is the "**Random Number**" which is represented in the form of hexadecimal: 0-9 for regular numbers and A - F (10 - 15) for hexadecimal characters.

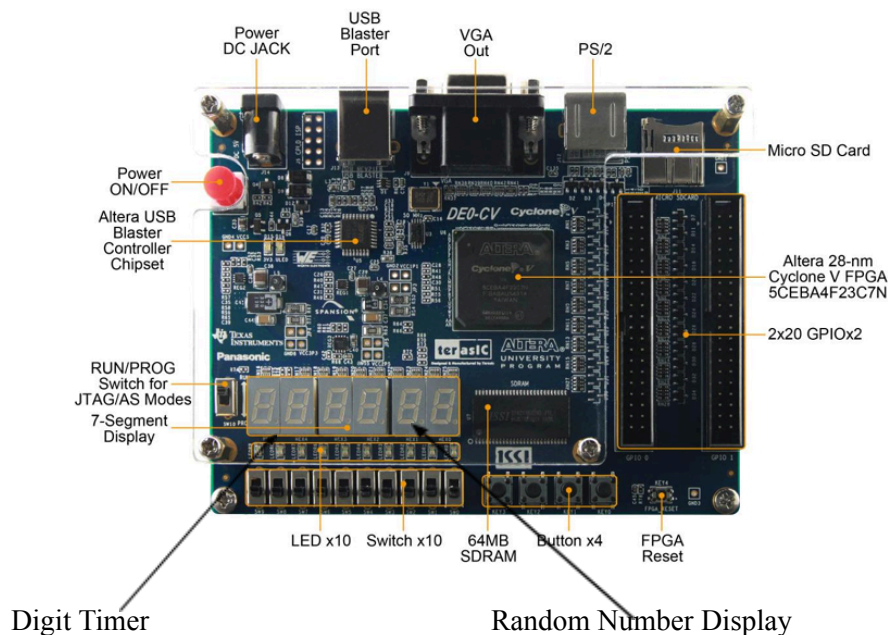


Figure 1.2 : Complete FPGA Board layout for Hardware based Number Generator

To begin the game, the user presses the **POWER ON/OFF** button to turn on the FPGA board. Next, the user will enter their correct password, which is **2062**. Looking from right to left, the last right 4 toggle switches - assuming the leftmost bit is the MSB and the rightmost bit is the LSB - are used to enter the password. First, 2 (0010) is inputted using the toggle switches, and the Password entering button is pressed. Next, 0(0000) is inputted using the toggle switches, and the Password entering button is pressed. Next, 6(0110) is inputted using the toggle switches, and the Password entering button is pressed. Finally, 2(0010) is inputted using the toggle switches, and the “**Logged In**” LED switches on. **In a case where the password input is incorrect, the user needs to try the entire process again.**

If the user is successful with entering the password, then the number “99” will display on the digit timer segment display. To officially begin the countdown to zero, the user will press the game start button (or KEY1) button again, and the timer will start ticking down all the way to zero. During this time, the player will toggle the four rightmost switches on the FPGA Board to come up with a binary number and the player will press the Player 1 load button (KEY2) button to load in his or her number. The number will display as a hexadecimal value. Next, the player will press the Random Number Generation Button and a random number, in the form of hexadecimal, will display on the Random Number Generator segment display. Finally, the sum will show in the sum display segment. Note that the sum is equal to the player’s loaded number plus the random number generated from pressing the random number generator button.

Now that the user has successfully pressed the **Game Start** button, to begin the game, Player 1 sets a 4-bit binary number using their toggle switches. We can assume that Player 1 might choose the number 0110 (which equals 6 in decimal). This number is then displayed on the **Player’s number**. Player 1 will then press the random number generator button shall observe this number and mentally calculate the binary number such that he or she subtract Player 1’s 4 bit binary number from 1111. Once entered, the **Random number** display shows the random number generated. Assume that the random number generator displays a 9 in hexadecimal. The FPGA then computes the sum of the two numbers. It calculates $0110 + 1001$, which equals 1111, and this result is displayed on the **Sum** display. Now, Player 1 checks if the sum displayed matches the expected result of 1111 (15). If the sum is correct, this round is considered a **successful attempt**. If the sum is incorrect, the round is considered a **failed attempt**. In this case, the round was successful.

However, if the random number generator generated a number such that the sum of Player 1’s number and the random number do not add up to 1111, then this is considered a failed attempt. For instance, if Player 1 toggled a 0110, and the random

number generated a 7, then the sum is **D** (13 in hexadecimal). **It's important to note that Player 1 can do these actions until the digit timer reaches zero.** The number of successes of the sum adding up to 1111 is dependent on the digit timer and luck.

***Important to note that the matching LEDs and non-matching LEDS are proof that the random number generated and the player's input can be the same or not.**